PROJETO DE BANCO DE DADOS

Revisão de Banco de Dados

Do modelo relacional às consultas SQL

Aula 2 · Sistemas de Informação · 2026

Começamos medindo o que a turma já lembra

Responda sem consultar:

- O que diferencia dado de informação?
- Para que servem PK e FK?
- Qual é a diferença entre WHERE e HAVING?
- Quando usar INNER JOIN ou LEFT JOIN?

5 min

Duplas: anotem uma resposta e uma dúvida.

A revisão segue o caminho do dado até a decisão

1

Modelar

entidades e relações

2

Proteger

chaves e restrições

3

Manipular

INSERT, UPDATE, DELETE

4

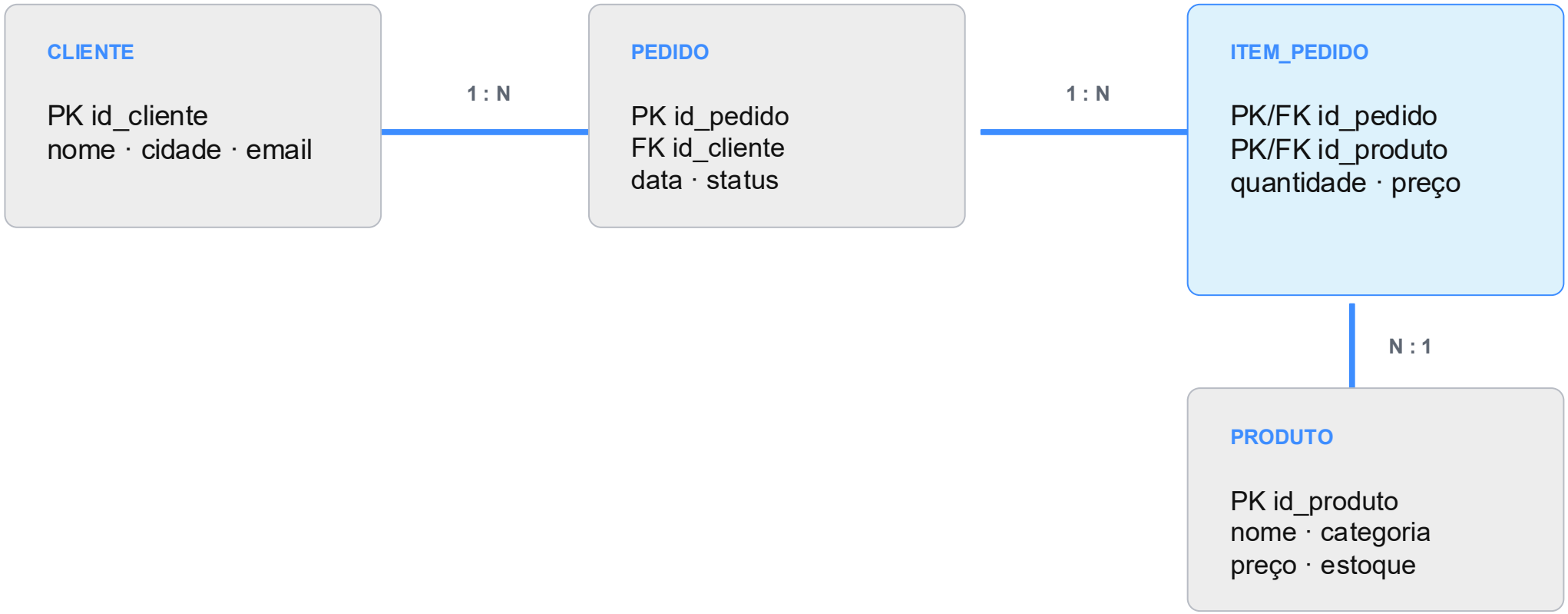
Consultar

SELECT, JOIN, GROUP BY

O SGBD organiza dados e controla o acesso

DADO	INFORMAÇÃO	BANCO DE DADOS	SGBD
“120,00”	**“Teclado custa R$ 120”**	**clientes + pedidos + produtos**	**SQLite, Oracle, PostgreSQL**
valor isolado	dado com contexto	coleção relacionada	software de gestão

Pedidos conectam clientes e produtos



Chaves e restrições impedem estados impossíveis

PRIMARY KEY	identifica cada linha	não pode repetir
FOREIGN KEY	mantém a referência	pedido exige cliente
NOT NULL	obriga um valor	produto exige nome
UNIQUE	evita duplicidade	email não se repete
CHECK	valida uma regra	preço > 0

NULL é ausência, não zero nem texto vazio

CORRETO

```
SELECT nome, email
FROM cliente
WHERE email IS NULL;
```

ARMADILHA

```
SELECT nome, email
FROM cliente
WHERE email = NULL;
```

Use IS NULL ou IS NOT NULL.

Comparações com NULL resultam em desconhecido.

SQL reúne comandos com responsabilidades diferentes

DDL	estrutura	`CREATE · ALTER · DROP`
DML	dados	`INSERT · UPDATE · DELETE`
DQL	consulta	`SELECT`
DCL	permissões	`GRANT · REVOKE`
TCL	transação	`COMMIT · ROLLBACK`

Prática SQL no navegador

ESTRUTURA

```
CREATE TABLE cliente (
  id_cliente INTEGER PRIMARY KEY,
  nome TEXT NOT NULL,
  cidade TEXT NOT NULL,
  email TEXT UNIQUE
);
```

dbfiddle.dev

1. Abra o editor
2. Selecione SQLite
3. Cole o script-base
4. Execute por blocos

INSERT transforma o esquema vazio em um cenário testável

INSERÇÃO

```
INSERT INTO produto VALUES
  (101, 'Teclado', 'Perifericos', 120, 15),
  (102, 'Mouse',   'Perifericos',  80, 30),
  (103, 'Monitor', 'Monitores',   950,  8);
```

Verifique imediatamente:

CHECK

```
SELECT *
FROM produto;
```

Inserir → consultar → validar

Cada cláusula do SELECT tem uma função

CONSULTA

```
SELECT nome, preco, estoque
FROM produto
WHERE preco >= 200 AND estoque > 0
ORDER BY preco DESC;
```

SELECT	quais colunas
FROM	qual origem
WHERE	quais linhas
ORDER BY	qual ordem

Mude o filtro para produtos abaixo de R$ 300.

JOIN recupera o significado espalhado entre tabelas

INNER JOIN

```
SELECT p.id_pedido, c.nome, p.status
FROM pedido AS p
JOIN cliente AS c
  ON c.id_cliente = p.id_cliente
ORDER BY p.id_pedido;
```

ON define como as linhas se conectam

Sem condição correta, o resultado pode multiplicar linhas.

LEFT JOIN preserva linhas sem par

LEFT JOIN

```
SELECT c.nome, COUNT(p.id_pedido) AS qtd
FROM cliente AS c
LEFT JOIN pedido AS p
  ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome;
```

INNER JOIN	somente linhas com par
LEFT JOIN	todas da esquerda
COUNT(p.id_pedido)	retorna zero sem pedido

Agregações resumem linhas; GROUP BY define o nível

GROUP BY + HAVING

```
SELECT c.nome,
       SUM(i.quantidade * i.preco_unitario) AS total
FROM cliente c
JOIN pedido p ON p.id_cliente = c.id_cliente
JOIN item_pedido i ON i.id_pedido = p.id_pedido
WHERE p.status = 'PAGO'
GROUP BY c.id_cliente, c.nome
HAVING SUM(i.quantidade * i.preco_unitario) >= 500;
```

WHERE filtra linhas antes

HAVING filtra grupos depois

Pergunta-chave:

“uma linha por quê?”

Altere dados com segurança

1	`SELECT`	teste o WHERE
2	`BEGIN`	abra a transação
3	`UPDATE / DELETE`	execute a mudança
4	`SELECT`	confira o resultado
5	`COMMIT ou ROLLBACK`	decida conscientemente

Sem WHERE, a alteração pode atingir todas as linhas.

Desafio: transforme perguntas em consultas

01 **Produtos com estoque, do menor para o maior preço**

02 **Clientes de Araraquara em ordem alfabética**

03 **Quantidade de produtos por categoria**

04 **Todos os clientes e sua quantidade de pedidos**

05 **Faturamento somente dos pedidos pagos**

06 **Produtos que nunca foram vendidos**

Duplas · prever resultado · escrever · executar · explicar

Como investigar erros de consulta

Sintoma	Causa provável	Como investigar
Linhas duplicadas	JOIN incompleto	revise cardinalidade e ON
Nenhuma linha	filtro forte ou NULL	teste condições separadas
Total alto demais	linhas multiplicadas	conte antes de somar
UPDATE amplo	WHERE ausente	SELECT com o mesmo filtro
FK recusada	referência inexistente	consulte a tabela-pai

FECHAMENTO

Uma boa consulta começa com um bom modelo

Modele relações · proteja integridade · consulte por etapas · valide o resultado

Saída: escreva uma dúvida que você resolveu e uma que ainda ficou.